

KarateFlow

Gestione e Tracciamento delle Performance Atletiche nel Karate

Simone Ingenito

Corso di Software Testing

27 Luglio 2026



1. Cos'è KarateFlow?

Piattaforma Web per il tracciamento atletico:

- Registrazione delle misurazioni fisiche e tecniche degli atleti.
- Generazione automatica di metriche di performance e report storici.

Esigenza Reale:

- Digitalizzare il lavoro dei preparatori, sostituendo i fogli cartacei o fogli Excel frammentati.

Value Proposition

Fornire ai Coach uno strumento digitale centralizzato per raccogliere test fisici e analizzare l'evoluzione prestazionale degli atleti.

1.5 Note sull'Autore: Studente e Atleta

La Doppia Carriera:

- Studente di Informatica Magistrale.
- Membro del programma Atleta Federiciano presso l'Università degli Studi di Napoli Federico II.
- Road to **EUSA Championships**, Tirana Settembre 2026.

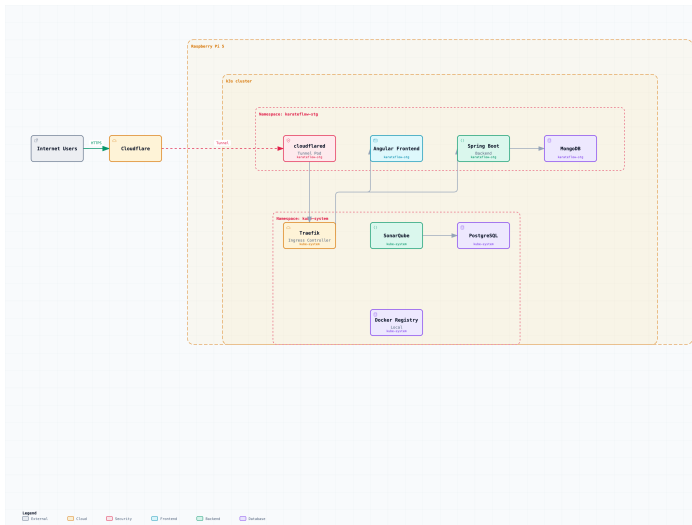
Il Collegamento con KarateFlow:

- Il software nasce da un'esigenza personale e del mio team di karate.



Campionati Nazionali Universitari, Maggio 2026

2. L'architettura: Server domestico e cluster K3s



3. L'applicazione (Ambiente di Staging)

L'applicazione web è interamente **hostata su un server Raspberry Pi**, fungendo da ambiente di Staging reale e accessibile dall'esterno.

Accesso alla Piattaforma

Link: staging.karate-flow.com

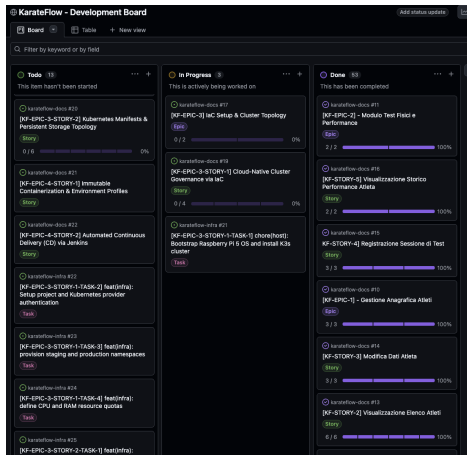
Principali funzionalità testabili in ambiente di staging:

- **Gestione Anagrafica:** È possibile registrare e aggiungere un nuovo atleta al sistema.
- **Registrazione Misurazioni:** Si possono creare e associare specifici test (tecnici o atletici) ai profili registrati.
- **Analisi e Confronti:** Aggiungendo **almeno due test** per un singolo atleta, l'applicazione sblocca la funzionalità di confronto, permettendo di generare un report visivo per analizzare l'andamento delle misurazioni nel tempo.

3.5 Tracciabilità e Gestione del Progetto

Per garantire un flusso di lavoro organizzato e tracciabile, ci si è affidati agli strumenti di **GitHub**:

Board di Progetto



Tracciabilità e PR

- **Issue e Milestone:** Organizzazione del lavoro in task specifici assegnati agli sviluppatori, raggruppati in Milestone per monitorare costantemente l'avanzamento.
- **Pull Request Template:** Adozione di template standard per assicurare che le modifiche al codice includano documentazione, test e rispettinno i requisiti prima del merge.

4. Contesto e Obiettivi del Corso di Software Testing

Il progetto KarateFlow, sviluppato inizialmente come evoluzione per il corso di **SPME**, viene integrato con una robusta suite di test e automazione per il corso di **Software Testing**.

I tre obiettivi principali del progetto di Software Testing sono:

- ❶ **Integrazione di Testing Avanzato:** Mutation Testing, Performance Load Testing, Automated E2E Testing.
- ❷ **Aumento della Coverage e Riduzione del Debito Tecnico:** Incremento della copertura dei branch ed eliminazione sistematica delle anomalie individuate dai linter e SonarQube.
- ❸ **Integrazione di GitHub Actions:** Realizzazione di pipeline in cloud e confronto con l'attuale strumento di CI/CD locale **Jenkins**.

4.4 Setup Iniziale: Clonazione dei Repository

Il progetto è suddiviso in più repository gestiti dall'organizzazione GitHub **KarateFlow**. Per configurare l'ambiente locale è necessario scaricarli tutti nella stessa cartella:

1. Creare la cartella principale:

```
mkdir KarateFlow && cd KarateFlow
```

2. Clonare i repository del progetto:

```
git clone https://github.com/KarateFlow/karateflow-be.git  
git clone https://github.com/KarateFlow/karateflow-fe.git  
git clone https://github.com/KarateFlow/karateflow-infra.git  
git clone https://github.com/KarateFlow/karateflow-docs.git
```

4.5 Esecuzione del progetto in locale (Docker-based)

L'intero ambiente di sviluppo locale è stato containerizzato, semplificando il setup ed eliminando la necessità di installare toolchain locali (Node, Maven, CLI).

L'avvio dell'applicazione richiede l'esecuzione sequenziale di due `docker-compose`:

- ❶ **Backend (Primo step):** Dalla cartella `karateflow-be`, `docker-compose up` avvia sia **MongoDB** che l'API in **Spring Boot**, montando il codice sorgente per il live-reload e abilitando il debugging remoto (porta 5005).
- ❷ **Frontend (Secondo step):** Successivamente, dalla cartella `karateflow-fe`, `docker-compose up` avvia l'app **Angular** tramite un container Node dedicato allo sviluppo, configurato per rilevare istantaneamente le modifiche al codice (polling).

Nota: Anche l'ambiente CI/CD (Jenkins) può essere avviato opzionalmente tramite Docker dalla cartella `karateflow-infra`.

4.6 Testing Suite (Backend & Frontend)

La strategia di Quality Assurance si divide in due stack tecnologici distinti:

Backend (Spring Boot / Java)

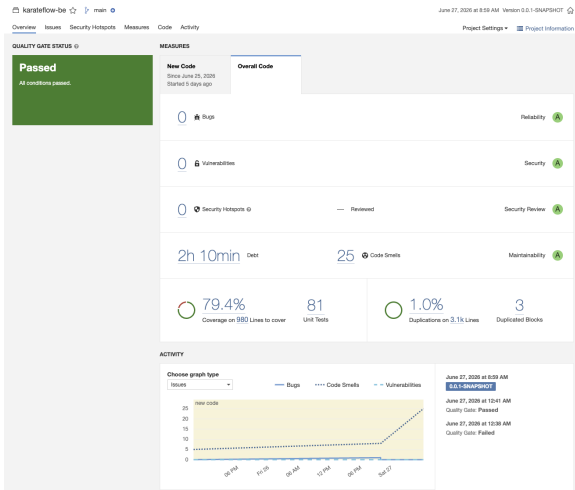
- **Unit & Integration Testing:** Sviluppati con **JUnit 5** e **Spring Boot Test** per testare unitariamente la logica e in integrazione le API REST.
- **Testcontainers:** Utilizzato per l'avvio di container effimeri di **MongoDB** durante i test di integrazione, garantendo un I/O reale rispetto ai database mockati.
- **Code Quality:** Linting del codice tramite **PMD** e code coverage misurata tramite **JaCoCo**.

Frontend (Angular / TypeScript)

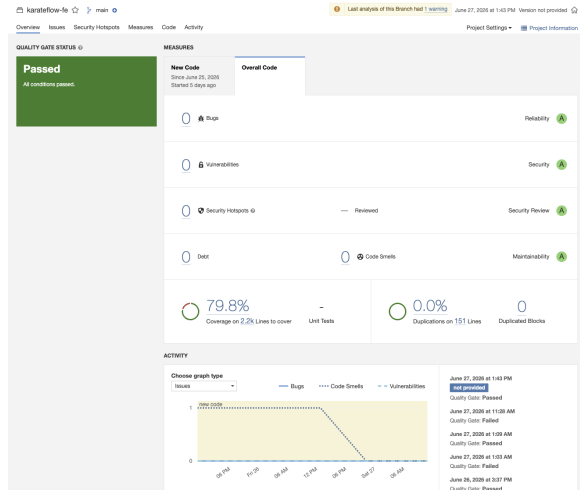
- **Unit Testing:** Adozione di **Vitest**, test runner moderno con report agganciati a SonarQube.
- **Code Quality:** Analisi statica tramite **ESLint**.

5. Stato iniziale della suite di test

Stato Iniziale Backend (SonarQube)



Stato Iniziale Frontend (SonarQube)



7. Performance & Load Testing

- *[Placeholder: Dettagli sugli strumenti utilizzati per i test di carico (es. JMeter, Gatling, k6)]*
- *[Placeholder: Risultati ottenuti (latenza, throughput, colli di bottiglia)]*
- *[Placeholder: Immagini o grafici dei risultati dei test di performance]*

8. Automated E2E Testing: Strumenti e Approccio

Per il testing End-to-End dell'applicazione web si è adottato un approccio **Behavior-Driven Development (BDD)** basato sulle seguenti tecnologie:

- **Cucumber:** Utilizzato per scrivere gli scenari di test in linguaggio naturale (Gherkin), garantendo una chiara definizione dei requisiti di business e un'elevata leggibilità.
- **Playwright:** Framework moderno impiegato "sotto il cofano" per l'automazione cross-browser. Consente di simulare in modo affidabile le interazioni reali dell'utente (es. compilazione dei form, click, navigazione).
- **Esecuzione:** La suite di test viene lanciata tramite il comando `npm run test:e2e` (che orchestra Playwright tramite cucumber-js).
- **Scenari scelti:** Ho implementato correttamente due scenari: Creazione dei test e Gestione Atleti.

8.1 Automated E2E Testing: Report di Esecuzione

 **cucumber-js@13.2.0 with node.js@26.0.0 on darwin@25.5.0** 

 **100% (2 / 2) passed**

 **6 seconds ago** *in 2.3 seconds*

 Search with text or @tags

Scenarios

>  e2e/features/creazione_test.feature

>  e2e/features/gestione_atleti.feature

Hooks

No test run hooks were executed.

9. DevOps: Stato attuale pipeline di CI/CD

- Attualmente gestito tramite **Jenkins**.

Tutto +

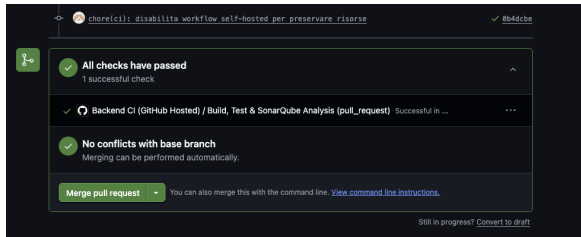
S	A	Nome ↓	Ultima compilazione completata con successo	Ultima compilazione non riuscita	Ultima durata	
✓	☁	CD Backend KarateFlow	1 g 5 h #8	12 g #5	23 s	▶
✓	☀	CD Frontend KarateFlow	4 g 7 h #16	5 g 9 h #10	26 s	▶
✓	☀	CI Backend KarateFlow	12 h #14	1 g 6 h #9	36 s	▶
✓	☁	CI Frontend KarateFlow	9 h 8 min #24	1 g 11 h #20	28 s	▶

- Jenkins** è in esecuzione sulla **macchina di sviluppo** per evitare di saturare le risorse del Raspberry Pi (server della piattaforma).

10. DevOps: Integrazione Github Actions

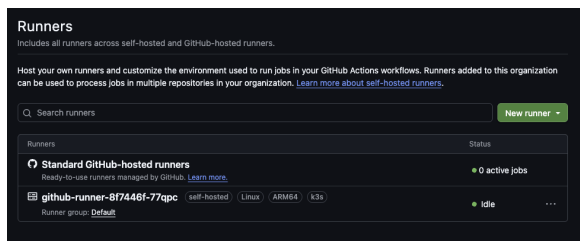
Per sfruttare l'infrastruttura cloud nativa, le **pipeline originariamente scritte per Jenkins sono state interamente tradotte in workflow di GitHub Actions**.

Workflow GitHub



The screenshot shows a GitHub Actions workflow named "chore(ci): disabilita workflow self-hosted per preservare risorse" with a green checkmark and commit hash "0b4dcbe". The workflow status is "All checks have passed" with "1 successful check". Below this, a green checkmark indicates "No conflicts with base branch" and "Merging can be performed automatically." A green button labeled "Merge pull request" is visible. At the bottom, it says "Still in progress? Convert to draft".

Runner Self-Hosted (Testato)



The screenshot shows the "Runners" management page in GitHub. It includes a search bar and a "New runner" button. Below, there are two sections: "Standard GitHub-hosted runners" showing "0 active jobs" and "github-runner-8f7446f-77qpc" which is "self-hosted" and "Idle". The runner group is set to "Default".

Runners	Status
Standard GitHub-hosted runners Ready-to-use runners managed by GitHub. Learn more.	0 active jobs
github-runner-8f7446f-77qpc self-hosted Linux ARM64 k3s Runner group: Default	Idle

11. DevOps: Github Actions vs Jenkins

- **Self-hosted vs GitHub-hosted runners:** Si è provato a usare runner self-hosted per GitHub Actions sul Raspberry Pi, ma saturavano le risorse impiegando troppo tempo. Si è preferito usare i server ospitati da GitHub.
- **Performance: Jenkins** (in esecuzione sulla macchina di sviluppo) risulta **leggermente più veloce** rispetto ai workflow eseguiti sui server GitHub.
- **Tempi di esecuzione:** Entrambe le soluzioni riescono comunque a concludere i task in **meno di un minuto**.
- **Trigger:** Jenkins viene avviato manualmente o con pooling mentre i workflow GHA si avviano manualmente ad ogni PR.

12. Prossime integrazioni

- Inclusione del **Mutation Testing** all'interno di una pipeline dedicata per le **Release**.
- Traduzione delle pipeline di CD in workflow Github Actions.
- *[Placeholder: Altre integrazioni future previste (Autenticazione, Observability e rilascio in Produzione).]*

13. Conclusioni

- *[Placeholder: Sintesi dei risultati ottenuti con il Software Testing]*
- *[Placeholder: Impatto della CI/CD sul processo di sviluppo]*
- *[Placeholder: Ringraziamenti o Q&A]*